

```
sudo less /path/to/kubelet-config.json
```

Verify that Webhook Authentication is enabled. This may be enabled as a command line argument to the kubelet service with `--authentication-token-webhook` or in the kubelet configuration file via `"authentication": { "webhook": { "enabled": true } }`.

Verify that the Authorization Mode is set to `WebHook`. This may be set as a command line argument to the kubelet service with `--authorization-mode=Webhook` or in the configuration file via `"authorization": { "mode": "Webhook" }`.

Audit Method 2:

It is also possible to review the running configuration of a Kubelet via the `/configz` endpoint of the Kubernetes API. This can be achieved using `kubectl` to proxy your requests to the API.

Discover all nodes in your cluster by running the following command:

```
kubectl get nodes
```

Next, initiate a proxy with `kubectl` on a local port of your choice. In this example we will use 8080:

```
kubectl proxy --port=8080
```

With this running, in a separate terminal run the following command for each node:

```
export NODE_NAME=my-node-name  
curl http://localhost:8080/api/v1/nodes/${NODE_NAME}/proxy/configz
```

The curl command will return the API response which will be a JSON formatted string representing the Kubelet configuration.

Verify that Webhook Authentication is enabled with `"authentication": { "webhook": { "enabled": true } }` in the API response.

Verify that the Authorization Mode is set to `WebHook` with `"authorization": { "mode": "Webhook" }` in the API response.

Remediation:

Remediation Method 1:

If configuring via the Kubelet config file, you first need to locate the file.

To do this, SSH to each node and execute the following command to find the kubelet process:

```
ps -ef | grep kubelet
```

The output of the above command provides details of the active kubelet process, from which we can see the location of the configuration file provided to the kubelet service with the `--config` argument. The file can be viewed with a command such as `more` or `less`, like so:

```
sudo less /path/to/kubelet-config.json
```

Enable Webhook Authentication by setting the following parameter:

```
"authentication": { "webhook": { "enabled": true } }
```

Next, set the Authorization Mode to **Webhook** by setting the following parameter:

```
"authorization": { "mode": "Webhook" }
```

Finer detail of the **authentication** and **authorization** fields can be found in the [Kubelet Configuration documentation](#).

Remediation Method 2:

If using executable arguments, edit the kubelet service file on each worker node and ensure the below parameters are part of the **KUBELET_ARGS** variable string.

For systems using **systemd**, such as the Amazon EKS Optimised Amazon Linux or Bottlerocket AMIs, then this file can be found at **/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf**. Otherwise, you may need to look up documentation for your chosen operating system to determine which service manager is configured:

```
--authentication-token-webhook  
--authorization-mode=Webhook
```

For Both Remediation Steps:

Based on your system, restart the **kubelet** service and check the service status.

The following example is for operating systems using **systemd**, such as the Amazon EKS Optimised Amazon Linux or Bottlerocket AMIs, and invokes the **systemctl** command. If **systemctl** is not available then you will need to look up documentation for your chosen operating system to determine which service manager is configured:

```
systemctl daemon-reload  
systemctl restart kubelet.service  
systemctl status kubelet -l
```

Default Value:

See the EKS documentation for the default value.

References:

1. <https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet/>
2. <https://kubernetes.io/docs/reference/access-authn-authz/kubelet-authn-authz/#kubelet-authentication>
3. <https://kubernetes.io/docs/reference/config-api/kubelet-config.v1beta1/>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	<u>5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts</u> Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.			
v7	<u>4.2 Change Default Passwords</u> Before deploying any new asset, change all default passwords to have values consistent with administrative level accounts.			

3.2.3 Ensure that a Client CA File is Configured (Automated)

Profile Applicability:

- Level 1

Description:

Enable Kubelet authentication using certificates.

Rationale:

The connections from the apiserver to the kubelet are used for fetching logs for pods, attaching (through kubectl) to running pods, and using the kubelet's port-forwarding functionality. These connections terminate at the kubelet's HTTPS endpoint. By default, the apiserver does not verify the kubelet's serving certificate, which makes the connection subject to man-in-the-middle attacks, and unsafe to run over untrusted and/or public networks. Enabling Kubelet certificate authentication ensures that the apiserver could authenticate the Kubelet before submitting any requests.

Impact:

You require TLS to be configured on apiserver as well as kubelets.

Audit:

Audit Method 1:

Kubelets can accept configuration via a configuration file and in some cases via command line arguments. It is important to note that parameters provided as command line arguments will override their counterpart parameters in the configuration file (see `--config` details in the [Kubelet CLI Reference](#) for more info, where you can also find out which configuration parameters can be supplied as a command line argument). With this in mind, it is important to check for the existence of command line arguments as well as configuration file entries when auditing Kubelet configuration. Firstly, SSH to each node and execute the following command to find the Kubelet process:

```
ps -ef | grep kubelet
```

The output of the above command provides details of the active Kubelet process, from which we can see the command line arguments provided to the process. Also note the location of the configuration file, provided with the `--config` argument, as this will be needed to verify configuration. The file can be viewed with a command such as `more` or `less`, like so:

```
sudo less /path/to/kubelet-config.json
```

Verify that a client certificate authority file is configured. This may be configured using a command line argument to the kubelet service with `--client-ca-file` or in the kubelet configuration file via `"authentication": { "x509": {"clientCAFile": <path/to/client-ca-file> } }`.

Audit Method 2:

It is also possible to review the running configuration of a Kubelet via the `/configz` endpoint of the Kubernetes API. This can be achieved using `kubectl` to proxy your requests to the API.

Discover all nodes in your cluster by running the following command:

```
kubectl get nodes
```

Next, initiate a proxy with `kubectl` on a local port of your choice. In this example we will use 8080:

```
kubectl proxy --port=8080
```

With this running, in a separate terminal run the following command for each node:

```
export NODE_NAME=my-node-name  
curl http://localhost:8080/api/v1/nodes/${NODE_NAME}/proxy/configz
```

The curl command will return the API response which will be a JSON formatted string representing the Kubelet configuration.

Verify that a client certificate authority file is configured with `"authentication": { "x509": {"clientCAFile": <path/to/client-ca-file> } }` in the API response.

Remediation:

Remediation Method 1:

If configuring via the Kubelet config file, you first need to locate the file.

To do this, SSH to each node and execute the following command to find the kubelet process:

```
ps -ef | grep kubelet
```

The output of the above command provides details of the active kubelet process, from which we can see the location of the configuration file provided to the kubelet service with the `--config` argument. The file can be viewed with a command such as `more` or `less`, like so:

```
sudo less /path/to/kubelet-config.json
```

Configure the client certificate authority file by setting the following parameter appropriately:

```
"authentication": { "x509": {"clientCAFile": <path/to/client-ca-file> } }"
```

Remediation Method 2:

If using executable arguments, edit the kubelet service file on each worker node and ensure the below parameters are part of the **KUBELET_ARGS** variable string.

For systems using **systemd**, such as the Amazon EKS Optimised Amazon Linux or Bottlerocket AMIs, then this file can be found at **/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf**. Otherwise, you may need to look up documentation for your chosen operating system to determine which service manager is configured:

```
--client-ca-file=<path/to/client-ca-file>
```

For Both Remediation Steps:

Based on your system, restart the **kubelet** service and check the service status.

The following example is for operating systems using **systemd**, such as the Amazon EKS Optimised Amazon Linux or Bottlerocket AMIs, and invokes the **systemctl** command. If **systemctl** is not available then you will need to look up documentation for your chosen operating system to determine which service manager is configured:

```
systemctl daemon-reload
systemctl restart kubelet.service
systemctl status kubelet -l
```

Default Value:

See the EKS documentation for the default value.

References:

1. <https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet/>
2. <https://kubernetes.io/docs/reference/access-authn-authz/kubelet-authn-authz/#kubelet-authentication>
3. <https://kubernetes.io/docs/reference/config-api/kubelet-config.v1beta1/>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 <u>Encrypt Sensitive Data in Transit</u> Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).		●	●
v7	14.4 <u>Encrypt All Sensitive Information in Transit</u> Encrypt all sensitive information in transit.		●	●

3.2.4 Ensure that the `--read-only-port` is disabled (Automated)

Profile Applicability:

- Level 1

Description:

Disable the read-only port.

Rationale:

The Kubelet process provides a read-only API in addition to the main Kubelet API. Unauthenticated access is provided to this read-only API which could possibly retrieve potentially sensitive information about the cluster.

Impact:

Removal of the read-only port will require that any service which made use of it will need to be re-configured to use the main Kubelet API.

Audit:

If using a Kubelet configuration file, check that there is an entry for `authentication: anonymous: enabled` set to `0`.

First, SSH to the relevant node:

Run the following command on each node to find the appropriate Kubelet config file:

```
ps -ef | grep kubelet
```

The output of the above command should return something similar to `--config /etc/kubernetes/kubelet/kubelet-config.json` which is the location of the Kubelet config file.

Open the Kubelet config file:

```
cat /etc/kubernetes/kubelet/kubelet-config.json
```

Verify that the `--read-only-port` argument exists and is set to `0`.

If the `--read-only-port` argument is not present, check that there is a Kubelet config file specified by `--config`. Check that if there is a `readOnlyPort` entry in the file, it is set to `0`.

Remediation:

If modifying the Kubelet config file, edit the `kubelet-config.json` file `/etc/kubernetes/kubelet/kubelet-config.json` and set the below parameter to `0`

```
"readOnlyPort": 0
```

If using executable arguments, edit the kubelet service file `/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf` on each worker node and add the below parameter at the end of the `KUBELET_ARGS` variable string.

```
--read-only-port=0
```

For each remediation:

Based on your system, restart the `kubelet` service and check status

```
systemctl daemon-reload
systemctl restart kubelet.service
systemctl status kubelet -l
```

Default Value:

See the Amazon EKS documentation for the default value.

References:

1. <https://kubernetes.io/docs/admin/kubelet/>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	12.6 <u>Use of Secure Network Management and Communication Protocols</u> Use secure network management and communication protocols (e.g., 802.1X, Wi-Fi Protected Access 2 (WPA2) Enterprise or greater).			
v7	9.2 <u>Ensure Only Approved Ports, Protocols and Services Are Running</u> Ensure that only network ports, protocols, and services listening on a system with validated business needs, are running on each system.			

3.2.5 Ensure that the `--streaming-connection-idle-timeout` argument is not set to 0 (Automated)

Profile Applicability:

- Level 1

Description:

Do not disable timeouts on streaming connections.

Rationale:

Setting idle timeouts ensures that you are protected against Denial-of-Service attacks, inactive connections and running out of ephemeral ports.

Note: By default, `--streaming-connection-idle-timeout` is set to 4 hours which might be too high for your environment. Setting this as appropriate would additionally ensure that such streaming connections are timed out after serving legitimate use cases.

Impact:

Long-lived connections could be interrupted.

Audit:

Audit Method 1:

First, SSH to the relevant node:

Run the following command on each node to find the running kubelet process:

```
ps -ef | grep kubelet
```

If the command line for the process includes the argument `streaming-connection-idle-timeout` verify that it is not set to 0.

If the `streaming-connection-idle-timeout` argument is not present in the output of the above command, refer instead to the `config` argument that specifies the location of the Kubelet config file e.g. `--config /etc/kubernetes/kubelet/kubelet-config.json`.

Open the Kubelet config file:

```
cat /etc/kubernetes/kubelet/kubelet-config.json
```

Verify that the **streamingConnectionIdleTimeout** argument is not set to 0.

Audit Method 2:

If using the api configz endpoint consider searching for the status of **"streamingConnectionIdleTimeout": "4h0m0s"** by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name;

HOSTNAME_PORT="localhost-and-port-number"

NODE_NAME="The-Name-Of-Node-To-Extract-Configuration" from the output of **"kubectl get nodes"**

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192.168.31.226.ec2.internal (example node name from
"kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"
```

Remediation:

Remediation Method 1:

If modifying the Kubelet config file, edit the kubelet-config.json file **/etc/kubernetes/kubelet/kubelet-config.json** and set the below parameter to a non-zero value in the format of **#h#m#s**

```
"streamingConnectionIdleTimeout": "4h0m0s"
```

You should ensure that the kubelet service file **/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf** does not specify a **--streaming-connection-idle-timeout** argument because it would override the Kubelet config file.

Remediation Method 2:

If using executable arguments, edit the kubelet service file **/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf** on each worker node and add the below parameter at the end of the **KUBELET_ARGS** variable string.

```
--streaming-connection-idle-timeout=4h0m0s
```

Remediation Method 3:

If using the api configz endpoint consider searching for the status of **"streamingConnectionIdleTimeout":** by extracting the live configuration from the nodes running kubelet.

****See detailed step-by-step configmap procedures in [Reconfigure a Node's Kubelet in a Live Cluster](#), and then rerun the curl statement from audit process to check for kubelet configuration changes**

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192.168.31.226.ec2.internal (example node name from
"kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"
```

For all three remediations:

Based on your system, restart the **kubelet** service and check status

```
systemctl daemon-reload
systemctl restart kubelet.service
systemctl status kubelet -l
```

Default Value:

See the EKS documentation for the default value.

References:

1. <https://kubernetes.io/docs/admin/kubelet/>
2. <https://github.com/kubernetes/kubernetes/pull/18552>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	12.6 <u>Use of Secure Network Management and Communication Protocols</u> Use secure network management and communication protocols (e.g., 802.1X, Wi-Fi Protected Access 2 (WPA2) Enterprise or greater).			
v7	9.2 <u>Ensure Only Approved Ports, Protocols and Services Are Running</u> Ensure that only network ports, protocols, and services listening on a system with validated business needs, are running on each system.			

3.2.6 Ensure that the `--make-iptables-util-chains` argument is set to true (Automated)

Profile Applicability:

- Level 1

Description:

Allow Kubelet to manage iptables.

Rationale:

Kubelets can automatically manage the required changes to iptables based on how you choose your networking options for the pods. It is recommended to let kubelets manage the changes to iptables. This ensures that the iptables configuration remains in sync with pods networking configuration. Manually configuring iptables with dynamic pod network configuration changes might hamper the communication between pods/containers and to the outside world. You might have iptables rules too restrictive or too open.

Impact:

Kubelet would manage the iptables on the system and keep it in sync. If you are using any other iptables management solution, then there might be some conflicts.

Audit:

Audit Method 1:

First, SSH to each node:

Run the following command on each node to find the Kubelet process:

```
ps -ef | grep kubelet
```

If the output of the above command includes the argument `--make-iptables-util-chains` then verify it is set to true.

If the `--make-iptables-util-chains` argument does not exist, and there is a Kubelet config file specified by `--config`, verify that the file does not set `makeIPTablesUtilChains` to `false`.

Audit Method 2:

If using the api configz endpoint consider searching for the status of `authentication... "makeIPTablesUtilChains.:true` by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name;

`HOSTNAME_PORT="localhost-and-port-number"`

`NODE_NAME="The-Name-Of-Node-To-Extract-Configuration"` from the output of `"kubectl get nodes"`

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192.168.31.226.ec2.internal (example node name from
"kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"
```

Remediation:

Remediation Method 1:

If modifying the Kubelet config file, edit the kubelet-config.json file `/etc/kubernetes/kubelet/kubelet-config.json` and set the below parameter to true

```
"makeIPTablesUtilChains": true
```

Ensure that `/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf` does not set the `--make-iptables-util-chains` argument because that would override your Kubelet config file.

Remediation Method 2:

If using executable arguments, edit the kubelet service file `/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf` on each worker node and add the below parameter at the end of the `KUBELET_ARGS` variable string.

```
--make-iptables-util-chains:true
```

Remediation Method 3:

If using the api configz endpoint consider searching for the status of `"makeIPTablesUtilChains.: true` by extracting the live configuration from the nodes running kubelet.

**See detailed step-by-step configmap procedures in [Reconfigure a Node's Kubelet in a Live Cluster](#), and then rerun the curl statement from audit process to check for kubelet configuration changes

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192.168.31.226.ec2.internal (example node name from
"kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"
```

For all three remediations:

Based on your system, restart the `kubelet` service and check status

```
systemctl daemon-reload
systemctl restart kubelet.service
systemctl status kubelet -l
```

Default Value:

See the Amazon EKS documentation for the default value.

References:

1. <https://kubernetes.io/docs/admin/kubelet/>
2. <https://kubernetes.io/docs/tasks/administer-cluster/reconfigure-kubelet/>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	12.3 Securely Manage Network Infrastructure Securely manage network infrastructure. Example implementations include version-controlled-infrastructure-as-code, and the use of secure network protocols, such as SSH and HTTPS.			
v7	11.1 Maintain Standard Security Configurations for Network Devices Maintain standard, documented security configuration standards for all authorized network devices.			

3.2.7 Ensure that the `--eventRecordQPS` argument is set to 0 or a level which ensures appropriate event capture (Automated)

Profile Applicability:

- Level 1

Description:

Security relevant information should be captured. The `eventRecordQPS` on the Kubelet configuration can be used to limit the rate at which events are gathered and sets the maximum event creations per second. Setting this too low could result in relevant events not being logged, however the unlimited setting of 0 could result in a denial of service on the kubelet.

Rationale:

It is important to capture all events and not restrict event creation. Events are an important source of security information and analytics that ensure that your environment is consistently monitored using the event data.

Impact:

Setting this parameter to 0 could result in a denial of service condition due to excessive events being created. The cluster's event processing and storage systems should be scaled to handle expected event loads.

Audit:

Run the following command on each node:

```
sudo grep "eventRecordQPS" /etc/systemd/system/kubelet.service.d/10-kubeadm.conf
```

Review the value set for the argument and determine whether this has been set to an appropriate level for the cluster.

If the argument does not exist, check that there is a Kubelet config file specified by `--config` and review the value in this location.

Remediation:

If using a Kubelet config file, edit the file to set `eventRecordQPS` to an appropriate level.

If using command line arguments, edit the kubelet service file `/etc/systemd/system/kubelet.service.d/10-kubeadm.conf` on each worker node and set the below parameter in `KUBELET_SYSTEM_PODS_ARGS` variable.

Based on your system, restart the `kubelet` service. For example:

```
systemctl daemon-reload
systemctl restart kubelet.service
```

Default Value:

See the Amazon EKS documentation for the default value.

References:

1. <https://kubernetes.io/docs/admin/kubelet/>
2. <https://github.com/kubernetes/kubernetes/blob/master/pkg/kubelet/apis/kubeletconfig/v1beta1/types.go>
3. <https://kubernetes.io/docs/tasks/administer-cluster/reconfigure-kubelet/>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	8.2 Collect Audit Logs Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.			
v8	8.5 Collect Detailed Audit Logs Configure detailed audit logging for enterprise assets containing sensitive data. Include event source, date, username, timestamp, source addresses, destination addresses, and other useful elements that could assist in a forensic investigation.			
v7	6.2 Activate audit logging Ensure that local logging has been enabled on all systems and networking devices.			
v7	6.3 Enable Detailed Logging Enable system logging to include detailed information such as an event source, date, user, timestamp, source addresses, destination addresses, and other useful elements.			

3.2.8 Ensure that the `--rotate-certificates` argument is not present or is set to true (Automated)

Profile Applicability:

- Level 1

Description:

Enable kubelet client certificate rotation.

Rationale:

The `--rotate-certificates` setting causes the kubelet to rotate its client certificates by creating new CSRs as its existing credentials expire. This automated periodic rotation ensures that there is no downtime due to expired certificates and thus addressing availability in the CIA (Confidentiality, Integrity, and Availability) security triad.

Note: This recommendation only applies if you let kubelets get their certificates from the API server. In case your kubelet certificates come from an outside authority/tool (e.g. Vault) then you need to implement rotation yourself.

Note: This feature also requires the `RotateKubeletClientCertificate` feature gate to be enabled.

Impact:

None

Audit:

Audit Method 1:

SSH to each node and run the following command to find the Kubelet process:

```
ps -ef | grep kubelet
```

If the output of the command above includes the `--RotateCertificate` executable argument, verify that it is set to true.

If the output of the command above does not include the `--RotateCertificate` executable argument then check the Kubelet config file. The output of the above command should return something similar to `--config /etc/kubernetes/kubelet/kubelet-config.json` which is the location of the Kubelet config file.

Open the Kubelet config file:

```
cat /etc/kubernetes/kubelet/kubelet-config.json
```

Verify that the `RotateCertificate` argument is not present, or is set to `true`.

Remediation:

Remediation Method 1:

If modifying the Kubelet config file, edit the kubelet-config.json file `/etc/kubernetes/kubelet/kubelet-config.json` and set the below parameter to true

```
"RotateCertificate":true
```

Additionally, ensure that the kubelet service file `/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf` does not set the `--RotateCertificate` executable argument to false because this would override the Kubelet config file.

Remediation Method 2:

If using executable arguments, edit the kubelet service file `/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf` on each worker node and add the below parameter at the end of the `KUBELET_ARGS` variable string.

```
--RotateCertificate=true
```

Default Value:

See the Amazon EKS documentation for the default value.

References:

1. <https://github.com/kubernetes/kubernetes/pull/41912>
2. <https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet-tls-bootstrapping/#kubelet-configuration>
3. <https://kubernetes.io/docs/imported/release/notes/>
4. <https://kubernetes.io/docs/reference/command-line-tools-reference/feature-gates/>
5. <https://kubernetes.io/docs/tasks/administer-cluster/reconfigure-kubelet/>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 <u>Encrypt Sensitive Data in Transit</u> Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).			
v7	14.4 <u>Encrypt All Sensitive Information in Transit</u> Encrypt all sensitive information in transit.			

3.2.9 Ensure that the RotateKubeletServerCertificate argument is set to true (Automated)

Profile Applicability:

- Level 1

Description:

Enable kubelet server certificate rotation.

Rationale:

RotateKubeletServerCertificate causes the kubelet to both request a serving certificate after bootstrapping its client credentials and rotate the certificate as its existing credentials expire. This automated periodic rotation ensures that there are no downtimes due to expired certificates and thus addressing availability in the CIA (Confidentiality, Integrity, and Availability) security triad.

Note: This recommendation only applies if you let kubelets get their certificates from the API server. In case your kubelet certificates come from an outside authority/tool (e.g. Vault) then you need to implement rotation yourself.

Impact:

None

Audit:

Audit Method 1:

First, SSH to each node:

Run the following command on each node to find the Kubelet process:

```
ps -ef | grep kubelet
```

If the output of the command above includes the **--rotate-kubelet-server-certificate** executable argument verify that it is set to true.

If the process does not have the **--rotate-kubelet-server-certificate** executable argument then check the Kubelet config file. The output of the above command should return something similar to **--config /etc/kubernetes/kubelet/kubelet-config.json** which is the location of the Kubelet config file.

Open the Kubelet config file:

```
cat /etc/kubernetes/kubelet/kubelet-config.json
```

Verify that **RotateKubeletServerCertificate** argument exists in the **featureGates** section and is set to **true**.

Audit Method 2:

If using the api configz endpoint consider searching for the status of

"RotateKubeletServerCertificate":true by extracting the live configuration from the nodes running kubelet.

Set the local proxy port and the following variables and provide proxy port number and node name;

HOSTNAME_PORT="localhost-and-port-number"

NODE_NAME="The-Name-Of-Node-To-Extract-Configuration" from the output of **"kubectl get nodes"**

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192.168.31.226.ec2.internal (example node name from
"kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"
```

Remediation:

Remediation Method 1:

If modifying the Kubelet config file, edit the kubelet-config.json file

/etc/kubernetes/kubelet/kubelet-config.json and set the below parameter to true

```
"featureGates": {
  "RotateKubeletServerCertificate":true
},
```

Additionally, ensure that the kubelet service file

/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf does not set the **--rotate-kubelet-server-certificate** executable argument to false because this would override the Kubelet config file.

Remediation Method 2:

If using executable arguments, edit the kubelet service file

/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf on each worker node and add the below parameter at the end of the **KUBELET_ARGS** variable string.

```
--rotate-kubelet-server-certificate=true
```

Remediation Method 3:

If using the api configz endpoint consider searching for the status of **"RotateKubeletServerCertificate"**: by extracting the live configuration from the nodes running kubelet.

**See detailed step-by-step configmap procedures in [Reconfigure a Node's Kubelet in a Live Cluster](#), and then rerun the curl statement from audit process to check for kubelet configuration changes

```
kubectyl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192.168.31.226.ec2.internal (example node name from
"kubectyl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"
```

For all three remediation methods:

Restart the **kubelet** service and check status. The example below is for when using systemctl to manage services:

```
systemctl daemon-reload
systemctl restart kubelet.service
systemctl status kubelet -l
```

Default Value:

See the Amazon EKS documentation for the default value.

References:

1. <https://github.com/kubernetes/kubernetes/pull/45059>
2. <https://kubernetes.io/docs/admin/kubelet-tls-bootstrapping/#kubelet-configuration>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 <u>Encrypt Sensitive Data in Transit</u> Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).			
v7	14.4 <u>Encrypt All Sensitive Information in Transit</u> Encrypt all sensitive information in transit.			

4 Policies

This section contains recommendations for various Kubernetes policies which are important to the security of Amazon EKS customer environment.

4.1 RBAC and Service Accounts

4.1.1 Ensure that the cluster-admin role is only used where required (Automated)

Profile Applicability:

- Level 1

Description:

The RBAC role `cluster-admin` provides wide-ranging powers over the environment and should be used only where and when needed.

Rationale:

Kubernetes provides a set of default roles where RBAC is used. Some of these roles such as `cluster-admin` provide wide-ranging privileges which should only be applied where absolutely necessary. Roles such as `cluster-admin` allow super-user access to perform any action on any resource. When used in a `ClusterRoleBinding`, it gives full control over every resource in the cluster and in all namespaces. When used in a `RoleBinding`, it gives full control over every resource in the rolebinding's namespace, including the namespace itself.

Impact:

Care should be taken before removing any `clusterrolebindings` from the environment to ensure they were not required for operation of the cluster. Specifically, modifications should not be made to `clusterrolebindings` with the `system:` prefix as they are required for the operation of system components.

Audit:

Obtain a list of the principals who have access to the `cluster-admin` role by reviewing the `clusterrolebinding` output for each role binding that has access to the `cluster-admin` role.

```
kubectl get clusterrolebindings -o=custom-
```

```
columns=NAME:.metadata.name,ROLE:.roleRef.name,SUBJECT:.subjects[*].name
```

Review each principal listed and ensure that `cluster-admin` privilege is required for it.

Remediation:

Identify all `clusterrolebindings` to the `cluster-admin` role. Check if they are used and if they need this role or if they could use a role with fewer privileges.

Where possible, first bind users to a lower privileged role and then remove the `clusterrolebinding` to the `cluster-admin` role :


```
kubectl delete clusterrolebinding [name]
```

Default Value:

By default a single **clusterrolebinding** called **cluster-admin** is provided with the **system:masters** group as its principal.

References:

1. <https://kubernetes.io/docs/admin/authorization/rbac/#user-facing-roles>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 <u>Restrict Administrator Privileges to Dedicated Administrator Accounts</u> Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.			
v7	4.3 <u>Ensure the Use of Dedicated Administrative Accounts</u> Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.			

4.1.2 Minimize access to secrets (Automated)

Profile Applicability:

- Level 1

Description:

The Kubernetes API stores secrets, which may be service account tokens for the Kubernetes API or credentials used by workloads in the cluster. Access to these secrets should be restricted to the smallest possible group of users to reduce the risk of privilege escalation.

Rationale:

Inappropriate access to secrets stored within the Kubernetes cluster can allow for an attacker to gain additional access to the Kubernetes cluster or external resources whose credentials are stored as secrets.

Impact:

Care should be taken not to remove access to secrets to system components which require this for their operation

Audit:

Review the users who have **get**, **list** or **watch** access to **secrets** objects in the Kubernetes API.

Remediation:

Where possible, remove **get**, **list** and **watch** access to **secret** objects in the cluster.

Default Value:

By default, the following list of principals have **get** privileges on **secret** objects

CLUSTERROLEBINDING	SUBJECT
TYPE SA-NAMESPACE	
cluster-admin	system:masters
Group	
system:controller:clusterrole-aggregation-controller	clusterrole-
aggregation-controller ServiceAccount kube-system	
system:controller:expand-controller	expand-controller
ServiceAccount kube-system	
system:controller:generic-garbage-collector	generic-garbage-
collector ServiceAccount kube-system	
system:controller:namespace-controller	namespace-controller
ServiceAccount kube-system	
system:controller:persistent-volume-binder	persistent-volume-
binder ServiceAccount kube-system	
system:kube-controller-manager	system:kube-controller-
manager User	

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.1 <u>Establish and Maintain a Secure Configuration Process</u> Establish and maintain a secure configuration process for enterprise assets (end-user devices, including portable and mobile, non-computing/IoT devices, and servers) and software (operating systems and applications). Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.			
v7	5.2 <u>Maintain Secure Images</u> Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.			

4.1.3 Minimize wildcard use in Roles and ClusterRoles (Automated)

Profile Applicability:

- Level 1

Description:

Kubernetes Roles and ClusterRoles provide access to resources based on sets of objects and actions that can be taken on those objects. It is possible to set either of these to be the wildcard "*" which matches all items.

Use of wildcards is not optimal from a security perspective as it may allow for inadvertent access to be granted when new resources are added to the Kubernetes API either as CRDs or in later versions of the product.

Rationale:

The principle of least privilege recommends that users are provided only the access required for their role and nothing more. The use of wildcard rights grants is likely to provide excessive rights to the Kubernetes API.

Audit:

Retrieve the roles defined across each namespaces in the cluster and review for wildcards

```
kubectl get roles --all-namespaces -o yaml
```

Retrieve the cluster roles defined in the cluster and review for wildcards

```
kubectl get clusterroles -o yaml
```

Remediation:

Where possible replace any use of wildcards in clusterroles and roles with specific objects or actions.

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.2 Use Unique Passwords Use unique passwords for all enterprise assets. Best practice implementation includes, at a minimum, an 8-character password for accounts using MFA and a 14-character password for accounts not using MFA.			

Controls Version	Control	IG 1	IG 2	IG 3
v7	4.4 Use Unique Passwords Where multi-factor authentication is not supported (such as local administrator, root, or service accounts), accounts will use passwords that are unique to that system.			

4.1.4 Minimize access to create pods (Automated)

Profile Applicability:

- Level 1

Description:

The ability to create pods in a namespace can provide a number of opportunities for privilege escalation, such as assigning privileged service accounts to these pods or mounting hostPaths with access to sensitive data (unless Pod Security Policies are implemented to restrict this access)

As such, access to create new pods should be restricted to the smallest possible group of users.

Rationale:

The ability to create pods in a cluster opens up possibilities for privilege escalation and should be restricted, where possible.

Impact:

Care should be taken not to remove access to pods to system components which require this for their operation

Audit:

Review the users who have create access to pod objects in the Kubernetes API.

Remediation:

Where possible, remove **create** access to **pod** objects in the cluster.

Default Value:

By default, the following list of principals have **create** privileges on **pod** objects

CLUSTERROLEBINDING	SUBJECT
TYPE SA-NAMESPACE	
cluster-admin	system:masters
Group	
system:controller:clusterrole-aggregation-controller	clusterrole-
aggregation-controller ServiceAccount kube-system	
system:controller:daemon-set-controller	daemon-set-controller
ServiceAccount kube-system	
system:controller:job-controller	job-controller
ServiceAccount kube-system	
system:controller:persistent-volume-binder	persistent-volume-
binder ServiceAccount kube-system	
system:controller:replicaset-controller	replicaset-controller
ServiceAccount kube-system	
system:controller:replication-controller	replication-controller
ServiceAccount kube-system	
system:controller:statefulset-controller	statefulset-controller
ServiceAccount kube-system	

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	2.7 <u>Allowlist Authorized Scripts</u> Use technical controls, such as digital signatures and version control, to ensure that only authorized scripts, such as specific .ps1, .py, etc., files, are allowed to execute. Block unauthorized scripts from executing. Reassess bi-annually, or more frequently.			●
v7	4.7 <u>Limit Access to Script Tools</u> Limit access to scripting tools (such as Microsoft PowerShell and Python) to only administrative or development users with the need to access those capabilities.		●	●

4.1.5 Ensure that default service accounts are not actively used. (Automated)

Profile Applicability:

- Level 1

Description:

The **default** service account should not be used to ensure that rights granted to applications can be more easily audited and reviewed.

Rationale:

Kubernetes provides a **default** service account which is used by cluster workloads where no specific service account is assigned to the pod.

Where access to the Kubernetes API from a pod is required, a specific service account should be created for that pod, and rights granted to that service account.

The default service account should be configured such that it does not provide a service account token and does not have any explicit rights assignments.

Impact:

All workloads which require access to the Kubernetes API will require an explicit service account to be created.

Audit:

For each namespace in the cluster, review the rights assigned to the default service account and ensure that it has no roles or cluster roles bound to it apart from the defaults.

Additionally ensure that the **automountServiceAccountToken: false** setting is in place for each default service account.

Remediation:

Create explicit service accounts wherever a Kubernetes workload requires specific access to the Kubernetes API server.

Modify the configuration of each default service account to include this value

```
automountServiceAccountToken: false
```

Automatic remediation for the default account:

```
kubectl patch serviceaccount default -p  
$'automountServiceAccountToken: false'
```


Default Value:

By default the **default** service account allows for its service account token to be mounted in pods in its namespace.

References:

1. <https://kubernetes.io/docs/tasks/configure-pod-container/configure-service-account/>
2. <https://aws.github.io/aws-eks-best-practices/iam/#disable-auto-mounting-of-service-account-tokens>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.3 <u>Disable Dormant Accounts</u> Delete or disable any dormant accounts after a period of 45 days of inactivity, where supported.			
v7	4.3 <u>Ensure the Use of Dedicated Administrative Accounts</u> Ensure that all users with administrative account access use a dedicated or secondary account for elevated activities. This account should only be used for administrative activities and not internet browsing, email, or similar activities.			
v7	16.9 <u>Disable Dormant Accounts</u> Automatically disable dormant accounts after a set period of inactivity.			

4.1.6 Ensure that Service Account Tokens are only mounted where necessary (Automated)

Profile Applicability:

- Level 1

Description:

Service accounts tokens should not be mounted in pods except where the workload running in the pod explicitly needs to communicate with the API server

Rationale:

Mounting service account tokens inside pods can provide an avenue for privilege escalation attacks where an attacker is able to compromise a single pod in the cluster.

Avoiding mounting these tokens removes this attack avenue.

Impact:

Pods mounted without service account tokens will not be able to communicate with the API server, except where the resource is available to unauthenticated principals.

Audit:

Review pod and service account objects in the cluster and ensure that the option below is set, unless the resource explicitly requires this access.

```
automountServiceAccountToken: false
```

Remediation:

Regularly review pod and service account objects in the cluster to ensure that the `automountServiceAccountToken` setting is `false` for pods and accounts that do not explicitly require API server access.

Default Value:

By default, all pods get a service account token mounted in them.

References:

1. <https://kubernetes.io/docs/tasks/configure-pod-container/configure-service-account/>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	<u>4.8 Uninstall or Disable Unnecessary Services on Enterprise Assets and Software</u> Uninstall or disable unnecessary services on enterprise assets and software, such as an unused file sharing service, web application module, or service function.			
v7	<u>14.7 Enforce Access Control to Data through Automated Tools</u> Use an automated tool, such as host-based Data Loss Prevention, to enforce access controls to data even when data is copied off a system.			

4.1.7 Cluster Access Manager API to streamline and enhance the management of access controls within EKS clusters (Automated)

Profile Applicability:

- Level 1

Description:

Amazon EKS has introduced the Cluster Access Manager API to streamline and enhance the management of access controls within EKS clusters. This new approach is now the recommended method over the traditional **aws-auth** ConfigMap for managing Role-Based Access Control (RBAC) and Service Accounts.

Key Advantages of Using the Cluster Access Manager API:

1. **Simplified Access Management:** The Cluster Access Manager API allows administrators to manage access directly through the Amazon EKS API, eliminating the need to modify the **aws-auth** ConfigMap manually. This reduces operational overhead and minimizes the risk of misconfigurations.
2. **Enhanced Security Controls:** With this API, administrators can assign predefined AWS-managed Kubernetes permissions, known as "access policies," to IAM principals. This provides a more secure and auditable way to manage permissions compared to manual ConfigMap edits.
3. **Improved Visibility and Auditing:** The API offers better visibility into cluster access configurations, facilitating easier auditing and compliance checks. Administrators can list and describe access entries and policies directly through the EKS API.

Rationale:

The compelling rationale for using the Cluster Access Manager API instead of the traditional **aws-auth** ConfigMap in Amazon EKS revolves around security, scalability, operational efficiency, and simplified management.

1. Increased Security and Reduced Risk

- **Direct Management via API:** The Cluster Access Manager API enables you to manage RBAC and IAM permissions directly through the EKS API rather than editing a ConfigMap. This eliminates the risk of inadvertent errors when manually modifying the **aws-auth** ConfigMap.
- **Immutable Access Entries:** The API ensures that once access entries are defined, they are tightly controlled, reducing the risk of accidental overwrites or misconfigurations that can happen when editing YAML files.
- **Fine-Grained Access Control:** By leveraging the new API, you can define access policies at a more granular level than the previous method. This ensures that only the necessary permissions are granted, minimizing the attack surface.

2. Operational Efficiency and Scalability

- **Scalability:** Managing access control through the aws-auth ConfigMap becomes increasingly challenging as the number of users and services grows. The new API scales better by allowing access management through standard AWS Identity and Access Management (IAM) tools.
- **Reduced Operational Overhead:** The API simplifies the management of access controls by removing the need for manual updates to the ConfigMap, reducing the risk of human error, and automating access provisioning through Infrastructure as Code (IaC) tools like Terraform or CloudFormation.

3. Improved Visibility, Auditing, and Compliance

- **Auditable and Traceable Changes:** The Cluster Access Manager API integrates with AWS CloudTrail, allowing you to track who made changes to access configurations. This level of visibility is critical for organizations that need to adhere to compliance frameworks like SOC 2, GDPR, or HIPAA.
- **Centralized Management:** Unlike the **aws-auth** ConfigMap, which is managed at the Kubernetes level, the new API leverages AWS IAM's centralized management and auditing capabilities, providing a unified view of access controls across your AWS environment.

4. Faster and Safer Access Provisioning

- **No More Cluster Downtime:** Errors in the aws-auth ConfigMap can accidentally lock out users or admins from the cluster, requiring complex recovery processes. The API-based approach is more resilient, reducing the risk of misconfigurations causing downtime.
- **Immediate Effect:** Changes made via the API take effect immediately, whereas updates to the aws-auth ConfigMap may require a delay or even restarting components in some cases.

5. Future-Proofing and Alignment with AWS Best Practices

- **Native Support in Kubernetes Versions:** Starting from Kubernetes 1.23, the Cluster Access Manager API is fully supported and designed to replace the aws-auth ConfigMap method. This aligns with AWS's roadmap and best practices for EKS, ensuring your infrastructure remains compatible with future updates.
- **Modern Approach for Pod Identity:** When combined with IAM Roles for Service Accounts (IRSA) or the new Pod Identity feature, the API supports a more dynamic and secure model for assigning permissions to pods, making it easier to implement least-privilege access.

Impact:

The shift to using the **Cluster Access Manager API** instead of the aws-auth ConfigMap impacts EKS RBAC and Service Accounts by simplifying access control management, reducing the risk of misconfigurations, and enhancing security. It allows for more granular, direct management of IAM permissions and Kubernetes roles, eliminating manual ConfigMap edits and reducing operational overhead. For Service Accounts, it better integrates with existing mechanisms like IAM Roles for Service Accounts (IRSA) for secure pod access to AWS resources, making it easier to enforce least-privilege principles. This transition improves scalability, auditing, and compliance, while providing a future-proof solution aligned with AWS's Kubernetes identity management roadmap.

Audit:

To check if the Cluster Access Manager API is active on your Amazon EKS cluster, you can use the following AWS CLI command:

```
aws eks describe-cluster --name $CLUSTER_NAME --query  
"cluster.accessConfig" --output json
```

Replace \$CLUSTER_NAME with the name of your EKS cluster.

The command queries the `cluster.accessConfig` property, which indicates the authentication mode of the cluster.

Possible Outputs:

If the output shows `"authenticationmode": "API"` or `"authenticationmode": "API_AND_CONFIG_MAP"`, it means the Cluster Access Manager API is enabled.

If it only shows `"authenticationmode": "CONFIG_MAP"`, then the cluster is still using the traditional `aws-auth` ConfigMap approach.

Remediation:

Log in to the AWS Management Console.

Navigate to Amazon EKS and select your EKS cluster.

Go to the Access tab and click on "Manage Access" in the "Access Configuration section".

Under Cluster Authentication Mode for Cluster Access settings.

- Click **EKS API** to change `cluster will source authenticated IAM principals only from EKS access entry APIs`.
- Click **ConfigMap** to change `cluster will source authenticated IAM principals only from the aws-auth ConfigMap`.
- Note: **EKS API** and **ConfigMap** must be selected during Cluster creation and cannot be changed once the Cluster is provisioned.

Default Value:

EKS API is selected by default during EKS Cluster creation but can be changed during initial configuration

References:

1. <https://aws.amazon.com/blogs/containers/a-deep-dive-into-simplified-amazon-eks-access-management-controls/>
2. <https://www.eksworkshop.com/docs/security/cluster-access-management/understanding>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	<u>4.8 Uninstall or Disable Unnecessary Services on Enterprise Assets and Software</u> Uninstall or disable unnecessary services on enterprise assets and software, such as an unused file sharing service, web application module, or service function.			
v7	<u>14.7 Enforce Access Control to Data through Automated Tools</u> Use an automated tool, such as host-based Data Loss Prevention, to enforce access controls to data even when data is copied off a system.			

4.1.8 Limit use of the Bind, Impersonate and Escalate permissions in the Kubernetes cluster (Manual)

Profile Applicability:

- Level 1

Description:

Cluster roles and roles with the impersonate, bind or escalate permissions should not be granted unless strictly required. Each of these permissions allow a particular subject to escalate their privileges beyond those explicitly granted by cluster administrators

Rationale:

The impersonate privilege allows a subject to impersonate other users gaining their rights to the cluster. The bind privilege allows the subject to add a binding to a cluster role or role which escalates their effective permissions in the cluster. The escalate privilege allows a subject to modify cluster roles to which they are bound, increasing their rights to that level.

Each of these permissions has the potential to allow for privilege escalation to cluster-admin level.

Impact:

There are some cases where these permissions are required for cluster service operation, and care should be taken before removing these permissions from system service accounts.

Audit:

Review the users who have access to cluster roles or roles which provide the impersonate, bind or escalate privileges.

Remediation:

Where possible, remove the impersonate, bind and escalate rights from subjects.

Default Value:

In a default kubeadm cluster, the system:masters group and clusterrole-aggregation-controller service account have access to the escalate privilege. The system:masters group also has access to bind and impersonate.

References:

1. <https://www.impidio.com/blog/kubernetes-rbac-security-pitfalls>
2. https://raesene.github.io/blog/2020/12/12/Escalating_Away/
3. <https://raesene.github.io/blog/2021/01/16/Getting-Into-A-Bind-with-Kubernetes/>

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	<u>4.8 Uninstall or Disable Unnecessary Services on Enterprise Assets and Software</u> Uninstall or disable unnecessary services on enterprise assets and software, such as an unused file sharing service, web application module, or service function.			
v7	<u>14.7 Enforce Access Control to Data through Automated Tools</u> Use an automated tool, such as host-based Data Loss Prevention, to enforce access controls to data even when data is copied off a system.			

4.2 Pod Security Standards

Pod Security Standards (PSS) are recommendations for securing deployed workloads to reduce the risks of container breakout. There are a number of ways if implementing PSS, including the built-in Pod Security Admission controller, or external policy control systems which integrate with Kubernetes via validating and mutating webhooks.

The previous feature described in this document, pod security policy (preview), was deprecated with version 1.21, and removed as of version 1.25. After pod security policy (preview) is deprecated, you must disable the feature on any existing clusters using the deprecated feature to perform future cluster upgrades and stay within Azure support.

4.2.1 Minimize the admission of privileged containers (Automated)

Profile Applicability:

- Level 1

Description:

Do not generally permit containers to be run with the `securityContext.privileged` flag set to `true`.

Rationale:

Privileged containers have access to all Linux Kernel capabilities and devices. A container running with full privileges can do almost everything that the host can do. This flag exists to allow special use-cases, like manipulating the network stack and accessing devices.

There should be at least one admission control policy defined which does not permit privileged containers.

If you need to run privileged containers, this should be defined in a separate policy and you should carefully check to ensure that only limited service accounts and users are given permission to use that policy.

Impact:

Pods defined with `spec.containers[].securityContext.privileged: true`, `spec.initContainers[].securityContext.privileged: true` and `spec.ephemeralContainers[].securityContext.privileged: true` will not be permitted.